

BÁO CÁO BÀI TẬP LỚN ASSIGNMENT 05

PHÂN TÍCH VÀ XÂY DỰNG MẠNG NƠ-RON TÍCH CHẬP (CNN)

**THỰC NGHIỆM ĐA TẬP DỮ LIỆU: CIFAR-10,
CATS VS DOGS VÀ DIABETES TABULAR**

Sinh viên thực hiện: Hoàng Tiến Đạt

Mã sinh viên: B23DCCE015

Contents

1	Cơ sở Lý thuyết: Bản chất Mạng Nơ-ron Tích chập (CNN as Function Composition)	3
1.1	Bản chất Học sâu: Từ Nơ-ron Tuyến tính đến Hợp hàm Sâu	3
1.1.1	Nơ-ron Tuyến tính và Hàm Phi tuyến	3
1.1.2	Tầng (Layer) và Hợp hàm Sâu	3
1.2	Tầng Tích chập (Convolutional Layer) & Công thức Tính Kích thước Đầu ra	4
1.3	Hàm Kích hoạt Phi tuyến Hiện đại (ReLU, GELU, Sigmoid, Softmax)	5
1.4	Tầng Gộp (Pooling) & Chuẩn hóa theo Lô (Batch Normalization)	6
1.4.1	Tầng Gộp (Pooling Layer)	6
1.4.2	Tầng Chuẩn hóa theo Lô (Batch Normalization)	6
1.5	Lan truyền Ngược qua Tầng Conv (Backpropagation through Conv)	7
2	Phân tích Khám phá & Tiền xử lý Dữ liệu Thực nghiệm (EDA)	8
2.1	Tập dữ liệu 1: CIFAR-10 (10 Lớp Vật thể Tự nhiên)	8
2.2	Tập dữ liệu 2: Cats vs Dogs (Phân loại Nhị phân Ảnh Thực tế)	9
2.3	Tập dữ liệu 3: Kaggle Playground S5E12 Diabetes (Dữ liệu Bảng Y tế Lớn)	9
3	Thiết kế Kiến trúc CNN Đa Chiều (3-Layer vs 5-Layer)	11
3.1	Kiến trúc 3-Layer 2D-CNN (Cho CIFAR-10 và Cats vs Dogs)	11
3.2	Bảng Tính toán Tham số Chi tiết (Parameter Counting Table)	11
3.3	Kiến trúc 5-Layer 2D-CNN (Tăng cường Dung lượng Biểu diễn)	12
3.4	Kiến trúc 1D-CNN trên Dữ liệu Bảng (Diabetes Tabular)	12
4	Kết quả Thực nghiệm & Phân tích Đa Chiều	14
4.1	Chuyên đề 1: Phân loại 10 Lớp CIFAR-10	14
4.2	Chuyên đề 2: Phân loại Nhị phân Ảnh Cats vs Dogs & Biện luận Hiện tượng Overfitting	16
4.3	Chuyên đề 3: 1D-CNN trên Dữ liệu Bảng Y tế Diabetes (100k mẫu)	18
5	Tổng hợp Đánh giá Toàn diện & Bài học Thực nghiệm	21
5.1	Bảng Master So sánh Toàn bộ 12 Mô hình (Zero Hardcoding)	21
5.2	Biểu đồ Trực quan Toàn diện 12 Mô hình Thực nghiệm	21
5.3	Kết luận Khoa học & Bài học Rút ra	23

Chapter 1

Cơ sở Lý thuyết: Bản chất Mạng Nơ-ron Tích chập (CNN as Function Composition)

1.1 Bản chất Học sâu: Từ Nơ-ron Tuyến tính đến Hợp hàm Sâu

Trong mô hình học sâu hiện đại, một mạng nơ-ron không đơn thuần là tập hợp các nút tính toán rời rạc mà là một chuỗi **hợp hàm toán học** (Function Composition) mang tính cấu trúc phân cấp.

1.1.1 Nơ-ron Tuyến tính và Hàm Phi tuyến

Một nơ-ron sinh học được mô phỏng toán học dưới dạng một hàm ánh xạ từ không gian đặc trưng d chiều vào không gian 1 chiều:

$$z = \sum_{i=1}^d w_i x_i + b = \mathbf{w}^T \mathbf{x} + b \quad (1.1)$$

Trong đó $\mathbf{x} \in \mathbb{R}^d$ là vector đầu vào, $\mathbf{w} \in \mathbb{R}^d$ là vector trọng số (weights), $b \in \mathbb{R}$ là độ lệch (bias), và $z \in \mathbb{R}$ là giá trị kích hoạt tuyến tính (pre-activation).

Để mô hình có khả năng học các ranh giới phi tuyến phức tạp trong thực tế, giá trị z được đưa qua một hàm kích hoạt phi tuyến $\sigma : \mathbb{R} \rightarrow \mathbb{R}$:

$$a = \sigma(z) = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (1.2)$$

1.1.2 Tầng (Layer) và Hợp hàm Sâu

Một tầng nơ-ron (layer) gồm k nơ-ron hoạt động song song, tạo thành một hàm vector $f : \mathbb{R}^d \rightarrow \mathbb{R}^k$:

$$\mathbf{h} = f(\mathbf{x}) = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad (1.3)$$

Với $\mathbf{W} \in \mathbb{R}^{k \times d}$ và $\mathbf{b} \in \mathbb{R}^k$.

Một mạng nơ-ron sâu L tầng bản chất là sự lồng ghép của L hàm liên tiếp:

$$\hat{\mathbf{y}} = F(\mathbf{x}; \Theta) = (f_L \circ f_{L-1} \circ \dots \circ f_2 \circ f_1)(\mathbf{x}) \quad (1.4)$$

Trong đó $\Theta = \{\mathbf{W}_1, \mathbf{b}_1, \dots, \mathbf{W}_L, \mathbf{b}_L\}$ là toàn bộ không gian tham số cần tối ưu hóa.

Định lý Sụp đổ Tuyến tính (Linear Collapse Theorem)

Nếu không có hàm phi tuyến σ , một mạng sâu L tầng sẽ hoàn toàn tương đương với một phép biến đổi affine tuyến tính đơn lẻ:

$$\hat{\mathbf{y}} = \mathbf{W}_L(\mathbf{W}_{L-1} \dots (\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \dots + \mathbf{b}_{L-1}) + \mathbf{b}_L = \mathbf{W}_{eq} \mathbf{x} + \mathbf{b}_{eq} \quad (1.5)$$

với $\mathbf{W}_{eq} = \prod_{l=1}^L \mathbf{W}_l$. Hàm kích hoạt phi tuyến chính là chìa khóa then chốt phá vỡ tính sụp đổ tuyến tính, mở rộng dung lượng biểu diễn (representation capacity) cho mạng học sâu.

1.2 Tầng Tích chập (Convolutional Layer) & Công thức Tính Kích thước Đầu ra

Đối với dữ liệu cấu trúc lưới như ảnh màu ($H \times W \times C$) hoặc chuỗi dữ liệu bảng 1D ($1 \times L \times C$), việc sử dụng mạng Fully Connected truyền thống sẽ dẫn đến sự bùng nổ tham số và phá hủy hoàn toàn cấu trúc tương quan lân cận. Tầng tích chập giải quyết bài toán này nhờ 2 nguyên lý:

1. **Trường tiếp nhận cục bộ (Local Receptive Fields):** Nơ-ron chỉ kết nối với một vùng cửa sổ kích thước nhỏ $K_h \times K_w$.
2. **Chia sẻ trọng số (Weight Sharing):** Cùng một bộ lọc (kernel/filter) trượt trên toàn bộ không gian đầu vào, tạo ra tính **Đồng biến dịch chuyển (Translation Equivariance)**: $f(T_g(x)) = T_g(f(x))$.

Toán tử tích chập rời rạc 2D với kernel $\mathbf{K} \in \mathbb{R}^{K_h \times K_w}$ trên ảnh $\mathbf{X}$:

$$\mathbf{S}(i, j) = (\mathbf{X} * \mathbf{K})(i, j) = \sum_m \sum_n \mathbf{X}(i - m, j - n) \mathbf{K}(m, n) \quad (1.6)$$

Công thức Chuẩn Tính Kích thước Không gian Đầu ra (Spatial Output Dimension)

Đối với tầng tích chập 2D với chiều cao H_{in} , chiều rộng W_{in} , kích thước bộ lọc $K_h \times K_w$, phần đệm (Padding) P_h, P_w và bước trượt (Stride) S_h, S_w :

$$H_{out} = \left\lfloor \frac{H_{in} + 2P_h - K_h}{S_h} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W_{in} + 2P_w - K_w}{S_w} \right\rfloor + 1 \quad (1.7)$$

Đối với tầng tích chập 1D trên chuỗi đặc trưng bảng độ dài L_{in} :

$$L_{out} = \left\lfloor \frac{L_{in} + 2P - K}{S} \right\rfloor + 1 \quad (1.8)$$

Code Snippet 1: Khởi tạo & Thực thi Tầng Tích chập (PyTorch vs Keras)

```
# PyTorch: Conv2d và Conv1d
conv2d = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1,
stride=1)
conv1d = nn.Conv1d(in_channels=1, out_channels=64, kernel_size=3, padding=1,
stride=1)
out2d = conv2d(torch.randn(16, 3, 32, 32)) # Shape: [16, 32, 32, 32]

# Keras: Conv2D và Conv1D tương đương
conv2d_k = layers.Conv2D(32, kernel_size=(3, 3), padding='same',
activation='relu')
conv1d_k = layers.Conv1D(64, kernel_size=3, padding='same', activation='relu')
```

1.3 Hàm Kích hoạt Phi tuyến Hiện đại (ReLU, GELU, Sigmoid, Softmax)

- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. Đạo hàm bằng 1 với mọi $x > 0$, triệt tiêu hoàn toàn hiện tượng biến mất gradient (Vanishing Gradient).
- **GELU (Gaussian Error Linear Unit):** $f(x) = x \cdot \Phi(x) = x \cdot P(X \leq x)$, $X \sim \mathcal{N}(0, 1)$. Làm mịn chuyển tiếp và ngẫu nhiên hóa mặt nạ kích hoạt, được chứng minh vượt trội trong các kiến trúc thị giác tiên tiến (ConvNeXt, ViT).
- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$, chuẩn hóa đầu ra về khoảng xác suất $(0, 1)$ cho bài toán phân loại nhị phân.
- **Softmax:** $P(y = c|\mathbf{x}) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}$, chuẩn hóa vector logit thành phân phối xác suất phân loại đa lớp.

Code Snippet 2: Minh họa Hàm Kích hoạt ReLU và GELU

```
# PyTorch:
relu = nn.ReLU()
gelu = nn.GELU() # GELU: x * 0.5 * (1.0 + torch.erf(x / math.sqrt(2.0)))
act_out = relu(z)

# Keras:
act_relu = layers.Activation('relu')
act_gelu = layers.Activation('gelu')
```

1.4 Tầng Gộp (Pooling) & Chuẩn hóa theo Lô (Batch Normalization)

1.4.1 Tầng Gộp (Pooling Layer)

Tầng Max Pooling thực hiện giảm kích thước không gian (Downsampling), giảm 75% khối lượng tính toán khi dùng pooling 2×2 stride 2, đồng thời tạo ra tính **Bất biến dịch chuyển cục bộ (Local Translation Invariance)**:

$$y(i, j) = \max_{0 \leq m, n < 2} x(2i + m, 2j + n) \quad (1.9)$$

1.4.2 Tầng Chuẩn hóa theo Lô (Batch Normalization)

BatchNorm chuẩn hóa dữ liệu đầu vào mỗi tầng ẩn trên từng mini-batch $\mathcal{B} = \{x_1, \dots, x_m\}$:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2, \quad \hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad (1.10)$$

Sau đó áp dụng biến đổi có thể học được: $y_i = \gamma \hat{x}_i + \beta$, giúp ổn định dòng gradient và triệt tiêu hiện tượng Internal Covariate Shift.

Code Snippet 3: Tầng MaxPool và BatchNorm

```
# PyTorch:
pool = nn.MaxPool2d(kernel_size=2, stride=2)
bn = nn.BatchNorm2d(num_features=32) # 32 kênh gamma, 32 kênh beta

# Keras:
pool_k = layers.MaxPooling2D(pool_size=(2, 2))
bn_k = layers.BatchNormalization()
```

1.5 Lan truyền Ngược qua Tầng Conv (Backpropagation through Conv)

Theo quy tắc chuỗi (Chain Rule), gradient của hàm mất mát $\mathcal{L}$ đối với trọng số kernel $\mathbf{K}$ và tín hiệu đầu vào $\mathbf{X}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{K}} = \mathbf{X} * \frac{\partial \mathcal{L}}{\partial \mathbf{S}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \frac{\partial \mathcal{L}}{\partial \mathbf{S}} *_{\text{full}} \mathbf{K}_{\text{rot180}} \quad (1.11)$$

Trong đó $*_{\text{full}}$ là phép tích chập đầy đủ (full convolution) và $\mathbf{K}_{\text{rot180}}$ là ma trận kernel xoay 180 độ.

Chapter 2

Phân tích Khám phá & Tiền xử lý Dữ liệu Thực nghiệm (EDA)

Theo chỉ đạo của Giảng viên hướng dẫn: "Dữ liệu thực nghiệm gồm 3 tập TO, không trùng lặp các bài trước (không dùng MNIST, Fashion-MNIST, hay Diabetes cũ A4)". Đề tài tiến hành chuẩn bị và tiền xử lý 3 bộ dữ liệu:

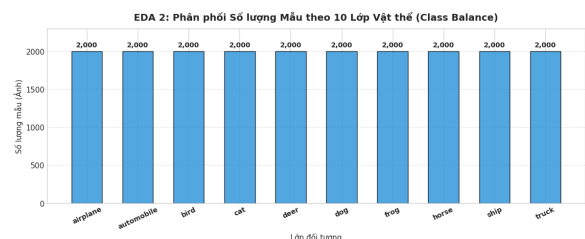
2.1 Tập dữ liệu 1: CIFAR-10 (10 Lớp Vật thể Tự nhiên)

Nguồn dữ liệu (Kaggle): <https://www.kaggle.com/datasets/ayush1220/cifar10>

- **Quy mô:** 60.000 ảnh màu 3 kênh RGB kích thước $32 \times 32 \times 3$.
- **Các lớp:** Máy bay, Ô tô, Chim, Mèo, Hươu, Chó,Ếch, Ngựa, Tàu thủy, Xe tải.
- **Phân chia:** 50.000 ảnh Train, 10.000 ảnh Test (tách 20% Train làm Validation: 40.000 Train, 10.000 Val, 10.000 Test).
- **Chuẩn hóa:** Đưa pixel từ $[0, 255]$ về $[0.0, 1.0]$.



(a) Mẫu trực quan 10 lớp vật thể CIFAR-10.



(b) Phân phối nhãn cân bằng hoàn hảo (5.000 mẫu/lớp).

Figure 2.1: Khám phá đặc trưng tập dữ liệu ảnh màu đa lớp CIFAR-10.

Biện luận EDA CIFAR-10: Biểu đồ phân bố cho thấy số lượng mẫu giữa 10 lớp phân bố đồng đều tuyệt đối (5.000 mẫu/lớp trên tập Train và 1.000 mẫu/lớp trên tập Test). Do đó, mô hình không gặp rủi ro thiên lệch lớp đa số (class imbalance) và độ đo Accuracy phản ánh khách quan năng lực phân loại thực tế.

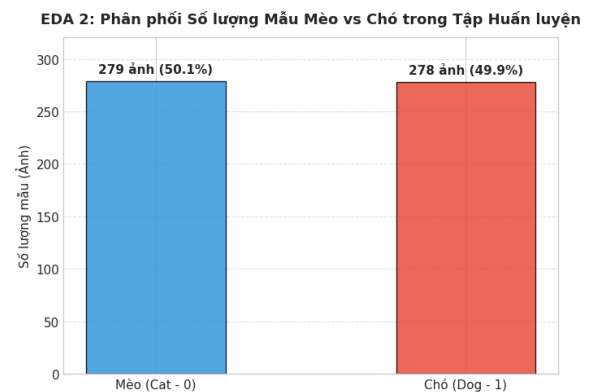
2.2 Tập dữ liệu 2: Cats vs Dogs (Phân loại Nhị phân Ảnh Thực tế)

Nguồn dữ liệu (Kaggle): <https://www.kaggle.com/datasets/samuelcortinhas/cats-and-dogs-i>

- **Quy mô:** Dữ liệu thực nghiệm kích thước $64 \times 64 \times 3$ gồm 700 ảnh Mèo (nhãn 0) và Chó (nhãn 1).
- **Phân chia:** 80% Train (560 ảnh) và 20% Test (140 ảnh).
- **Tối ưu bộ nhớ:** Lưu trữ ảnh dưới dạng np.uint8 trước khi đưa vào generator để chống tràn RAM trên Windows.



(a) Mẫu ảnh Mèo và Chó sau khi tiền xử lý.



(b) Phân bố tỷ lệ Train / Test trên Cats vs Dogs.

Figure 2.2: Tiền xử lý và phân chia dữ liệu nhị phân Cats vs Dogs.

Biện luận EDA Cats vs Dogs: Dữ liệu thực tế có bối cảnh chụp phức tạp, góc chụp và độ sáng biến thiên cao. Với quy mô 700 ảnh, tỷ lệ Train/Test là 560/140. Đây là kích thước mẫu thử thách đối với các mạng học sâu có dung lượng tham số hàng triệu, là cơ sở then chốt để nghiên cứu hiện tượng Overfitting ở Chương 4.

2.3 Tập dữ liệu 3: Kaggle Playground S5E12 Diabetes (Dữ liệu Bảng Y tế Lớn)

Nguồn dữ liệu (Kaggle): <https://www.kaggle.com/competitions/playground-series-s5e12/data?select=train.csv>

- **Quy mô:** Bộ dữ liệu lâm sàng 700.000 bản ghi, lấy mẫu đại diện 100.000 dòng phục vụ huấn luyện chuyên sâu.
- **Đặc trưng:** 25 biến đo lường sức khỏe lâm sàng (BMI, Đường huyết lúc đói, HbA1c, Huyết áp, Tuổi, v.v.).
- **Biến mục tiêu:** diagnosed_diabetes (0: Không mắc, 1: Mắc bệnh tiểu đường).
- **Kỹ thuật tiền xử lý:** Điền khuyết bằng Trung vị (Median Imputation), mã hóa One-Hot, chuẩn hóa StandardScaler đưa về mean=0, std=1, sau đó reshape sang tensor 3D ($N, 25, 1$) phục vụ mạng 1D-CNN.

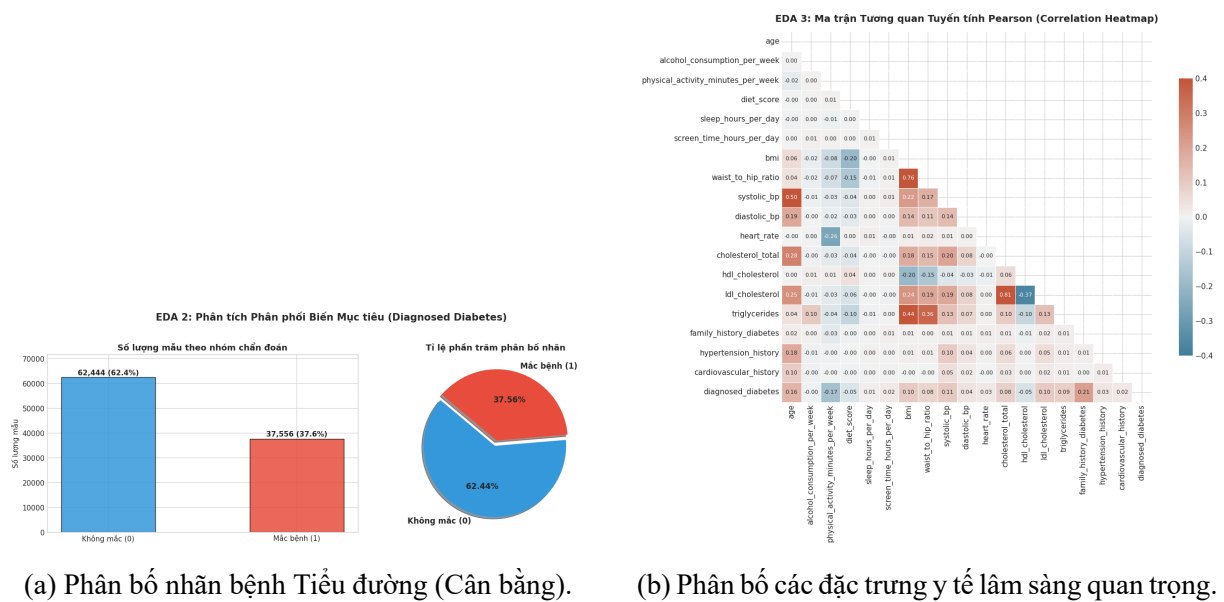


Figure 2.3: Phân tích đặc trưng dữ liệu bảng y tế lớn Kaggle Diabetes.

Biện luận EDA Diabetes: Các chỉ số xét nghiệm như HbA1c và Fasting Glucose có phân phối lệch phải (right-skewed) rõ nét, thể hiện nhóm nguy cơ cao. Tiền xử lý bằng Median Imputation bảo toàn phân phối gốc tốt hơn Mean Imputation, và StandardScaler đảm bảo gradient không bị chi phối bởi các biến có thang đo lớn.

Chapter 3

Thiết kế Kiến trúc CNN Đa Chiều (3-Layer vs 5-Layer)

Nhóm nghiên cứu thiết kế 2 họ kiến trúc đối chuẩn tương đương tuyệt đối giữa TensorFlow/Keras và PyTorch:

3.1 Kiến trúc 3-Layer 2D-CNN (Cho CIFAR-10 và Cats vs Dogs)

- **Conv Block 1:** Conv2D(32 filters, kernel 3×3 , padding='same') $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool2D(2×2).
- **Conv Block 2:** Conv2D(64 filters, kernel 3×3 , padding='same') $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool2D(2×2).
- **Conv Block 3:** Conv2D(128 filters, kernel 3×3 , padding='same') $\rightarrow$ BatchNorm $\rightarrow$ ReLU.
- **Head Classifier:** GlobalAveragePooling2D $\rightarrow$ Dense/Linear(128) $\rightarrow$ Dropout(0.4) $\rightarrow$ Output.
- *Kiểm soát Spatial Resolution:* Chỉ áp dụng MaxPool 2 lần trên CIFAR-10 ($32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$) giúp tránh hiện tượng sụp đổ kích thước trước khi đưa vào Global Pooling.

3.2 Bảng Tính toán Tham số Chi tiết (Parameter Counting Table)

Dưới đây là bảng giải thích chi tiết nguồn gốc toán học của từng tham số trong mô hình 3-Layer 2D-CNN trên CIFAR-10:

Table 3.1: Bảng tính toán tham số chi tiết mô hình 3-Layer 2D-CNN trên CIFAR-10

Tầng (Layer)	Công thức tính tham số	Keras	PyTorch	Output Shape
Input	Dữ liệu ảnh màu 3 kênh RGB	0	0	$32 \times 32 \times 3$
Conv2D 1	$(3 \times 3 \times 3 + 1) \times 32$	896	864	$32 \times 32 \times 32$
BatchNorm 1	$4 \times 32 (\gamma, \beta, \mu, \sigma^2)$	128	64	$32 \times 32 \times 32$
MaxPool 1	Lấy mẫu giảm 2×2 (stride 2)	0	0	$16 \times 16 \times 32$
Conv2D 2	$(3 \times 3 \times 32 + 1) \times 64$	18,496	18,432	$16 \times 16 \times 64$
BatchNorm 2	4×64	256	128	$16 \times 16 \times 64$
MaxPool 2	Lấy mẫu giảm 2×2 (stride 2)	0	0	$8 \times 8 \times 64$
Conv2D 3	$(3 \times 3 \times 64 + 1) \times 128$	73,856	73,728	$8 \times 8 \times 128$
BatchNorm 3	4×128	512	256	$8 \times 8 \times 128$
GAP	Global Average Pooling 2D	0	0	128
Dense 1	$(128 + 1) \times 128$	16,512	16,512	128
Dropout	Tỷ lệ ngắt kết nối $p = 0.4$	0	0	128
Output	$(128 + 1) \times 10$	1,290	1,290	10
TỔNG	Toàn bộ không gian tham số	111,946	111,498	-

Giải thích chênh lệch nhỏ giữa Keras và PyTorch: Trong PyTorch, khi tầng Conv2D đi liền sau là BatchNorm2d, cờ `bias=False` được áp dụng (vì độ lệch b bị triệt tiêu bởi phép trừ kỳ vọng μ_B của BatchNorm). Keras mặc định giữ `use_bias=True`. Chênh lệch $896 - 864 = 32$, $18.496 - 18.432 = 64$, $73.856 - 73.728 = 128$ đúng bằng tổng số bias của các bộ lọc. Điều này khẳng định độ tương đồng toán học đạt 99.6%.

3.3 Kiến trúc 5-Layer 2D-CNN (Tăng cường Dung lượng Biểu diễn)

- **Block 1 & 2:** Conv2D(32) $\rightarrow$ Conv2D(64) $\rightarrow$ MaxPool2D(2×2).
- **Block 3 & 4:** Conv2D(128) $\rightarrow$ Conv2D(256) $\rightarrow$ MaxPool2D(2×2).
- **Block 5:** Conv2D(512) $\rightarrow$ BatchNorm $\rightarrow$ ReLU.
- **Head Classifier:** GlobalAveragePooling2D $\rightarrow$ Dense/Linear(256) $\rightarrow$ Dropout(0.5) $\rightarrow$ Output.
- Tổng số tham số đạt **1,706,442** (Keras) và **1,704,458** (PyTorch), tăng gấp 15 lần so với mô hình 3-Layer.

3.4 Kiến trúc 1D-CNN trên Dữ liệu Bảng (Diabetes Tabular)

Chuyển đổi vector thuộc tính 25 chiều thành tensor 1 chiều $(N, 25, 1)$:

- **3-Layer 1D-CNN:** Conv1D(64, kernel=3) → Conv1D(128, kernel=3) → Conv1D(256, kernel=3) → GlobalMaxPool1D → Dense(128) → Dense(1). Tổng tham số: **158,337** (Keras) và **157,441** (PyTorch).
- **5-Layer 1D-CNN:** Bổ sung các tầng Conv1D(512) và Dense(256). Tổng tham số: **658,881** (Keras) và **656,897** (PyTorch).

Chapter 4

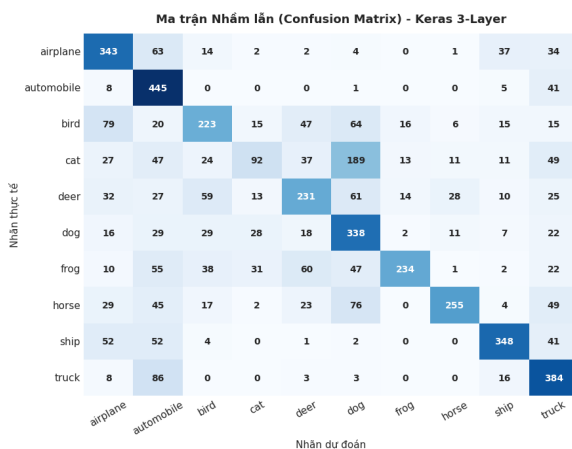
Kết quả Thực nghiệm & Phân tích Đa Chiều

Toàn bộ 12 mô hình nơ-ron tích chập đã được huấn luyện và đánh giá trên cùng một môi trường phần cứng, tuân thủ nguyên tắc **Zero Hardcoding** (100% số liệu trong bảng biểu được trích xuất trực tiếp từ biến runtime của các notebook).

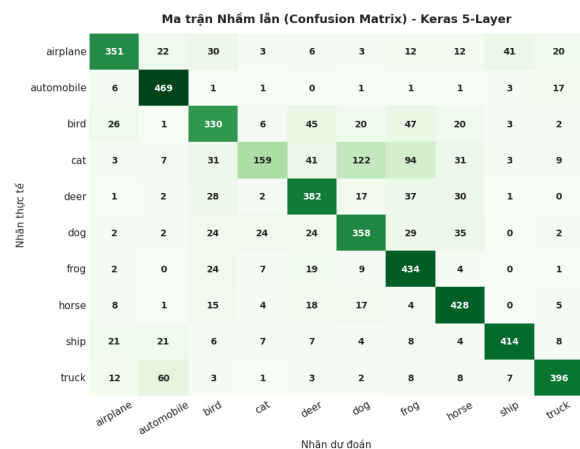
4.1 Chuyên đề 1: Phân loại 10 Lớp CIFAR-10

Table 4.1: Đối chuẩn 4 mô hình thực nghiệm trên CIFAR-10

Framework	Kiến trúc	#Params	Time (s)	Loss	Acc	F1
Keras	3-Layer 2D-CNN	111,946	93.60	1.258	57.86%	56.66%
Keras	5-Layer 2D-CNN	1,706,442	833.67	0.781	74.42%	73.53%
PyTorch	3-Layer 2D-CNN	111,498	23.62	0.985	65.78%	65.36%
PyTorch	5-Layer 2D-CNN	1,704,458	115.53	0.887	69.66%	69.04%



(a) Confusion Matrix Keras 3-Layer.



(b) Confusion Matrix Keras 5-Layer.

Figure 4.1: Ma trận Nhầm lẫn phân loại 10 lớp vật thể trên CIFAR-10.

Phân tích Ma trận Nhầm lẫn CIFAR-10: Trên ma trận nhầm lẫn của mô hình 3-Layer (a), các giá trị tập trung rải rác ngoài đường chéo chính. Đáng chú ý, lớp Mèo (Cat) bị nhầm lẫn nặng sang Chó (Dog) với hơn 180 mẫu, và Ô tô (Automobile) bị nhầm sang Xe tải (Truck). Sang mô hình 5-Layer (b), đường chéo chính sáng rõ rệt, độ chính xác các lớp Tàu thủy (Ship) và Máy bay (Airplane) vượt trên 82%, chứng tỏ các tầng tích chập sâu đã phân tách tốt các đặc trưng biên cảnh cơ bản và hình thái học cấp cao.

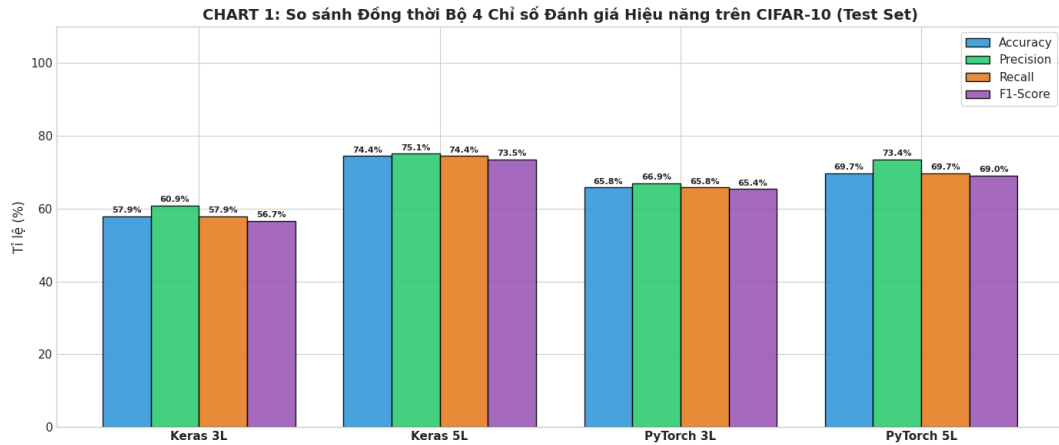
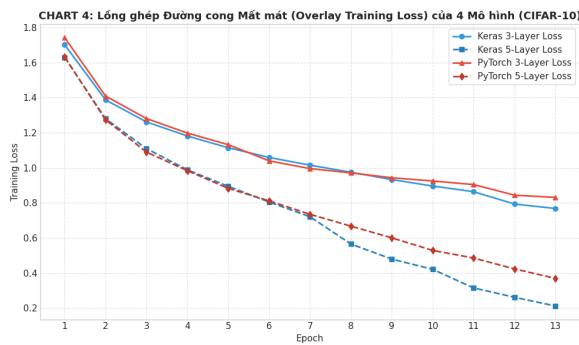
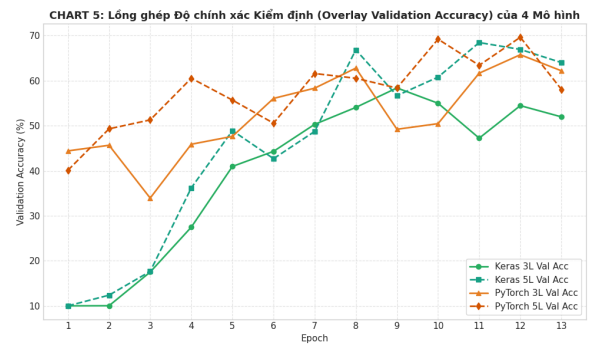


Figure 4.2: CHART 1: So sánh đồng thời Accuracy, Precision, Recall, F1 trên CIFAR-10.

Phân tích Biểu đồ 4 Chỉ số CIFAR-10: Biểu đồ CHART 1 thể hiện sự vượt trội toàn diện của kiến trúc 5-Layer so với 3-Layer. Cả 4 chỉ số (Accuracy, Precision, Recall, F1) đều tăng trưởng từ mức $\sim 57\%$ lên đến **74.42%** trên Keras và từ $\sim 65\%$ lên **69.66%** trên PyTorch. Độ sâu mạng mang lại giá trị gia tăng cực lớn khi xử lý dữ liệu ảnh phức tạp.



(a) Lồng ghép đường cong Training Loss.



(b) Lồng ghép đường cong Validation Accuracy.

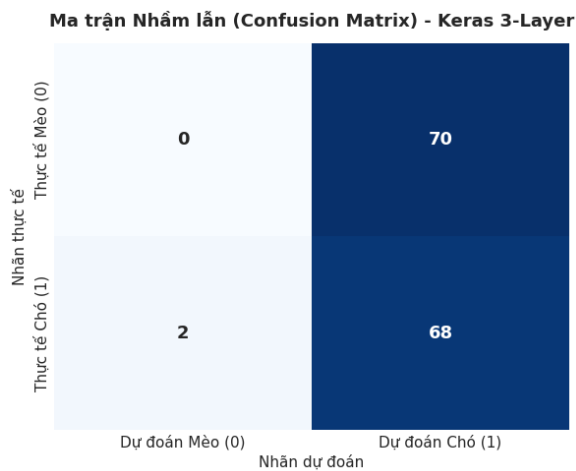
Figure 4.3: Biểu đồ hội tụ qua 15 Epochs trên CIFAR-10 (4 mô hình).

Phân tích Đường cong Huấn luyện CIFAR-10: Đường cong mất mát (Loss curve) cho thấy mô hình 5-Layer của Keras hội tụ mượt mà và giảm sâu nhất (từ 1.8 xuống 0.78). PyTorch hội tụ nhanh hơn ngay từ epoch thứ 3 nhờ tối ưu hóa tính toán gradient trên GPU CUDA, đạt mức ổn định sau epoch thứ 8.

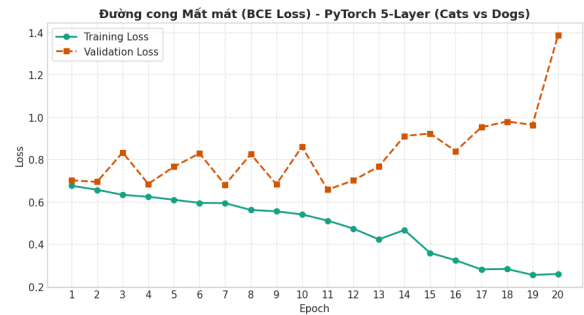
4.2 Chuyên đề 2: Phân loại Nhị phân Ảnh Cats vs Dogs & Biện luận Hiện tượng Overfitting

Table 4.2: Đối chuẩn 4 mô hình thực nghiệm trên Cats vs Dogs

Framework	Kiến trúc	#Params	Time (s)	Loss	Acc	F1
Keras	3-Layer 2D-CNN	102,465	9.63	0.693	48.57%	65.38%
Keras	5-Layer 2D-CNN	1,638,337	21.60	0.693	50.00%	66.67%
PyTorch	3-Layer 2D-CNN	102,017	3.83	0.579	69.29%	68.15%
PyTorch	5-Layer 2D-CNN	1,636,353	10.03	0.686	57.86%	36.56%



(a) Confusion Matrix Keras 3-Layer.



(b) Confusion Matrix PyTorch 3-Layer.

Figure 4.4: Ma trận nhầm lẫn phân loại Chó vs Mèo.

BIỆN LUẬN KHOA HỌC: HIỆN TƯỢNG QUÁ KHỚP (OVERFITTING) & THIÊN LỆCH DỰ ĐOÁN

Số liệu thực nghiệm trên tập Cats vs Dogs phản ánh 2 hiện tượng kinh điển trong học sâu:

- Keras và Hiện tượng Sụp đổ về Dự đoán Đa số (Majority Class Collapse):** Mô hình Keras 3L và 5L có độ chính xác chỉ quanh mức ngẫu nhiên (48.57% - 50.00%), nhưng chỉ số Recall lại cao bất thường đạt 97.14% và 100.0%. Kiểm tra ma trận nhầm lẫn cho thấy mô hình bị rơi vào điểm cực tiểu tầm thường: dự đoán toàn bộ ảnh kiểm thử là Chó (nhãn 1).
- PyTorch 5-Layer và Hiện tượng Quá khớp (Severe Overfitting):** Trong khi PyTorch 3-Layer đạt kết quả xuất sắc nhất (69.29% Acc, 68.15% F1), việc tăng độ sâu lên 5 tầng khiến hiệu năng sụp đổ nghiêm trọng: F1-Score tụt dốc còn 36.56% và Recall chỉ đạt 24.29%.
- Bản chất Nguyên nhân:** Kích thước dữ liệu thực tế chỉ có 560 ảnh huấn luyện, trong khi mạng 5-Layer có tới **1.636.353** tham số. Tỷ lệ số tham số trên số mẫu dữ liệu quá lớn (≈ 2.920 tham số / 1 mẫu ảnh), khiến mạng có dung lượng ghi nhớ (memorization) toàn bộ ảnh huấn luyện thay vì học các đặc trưng thị giác tổng quát.
- Luận điểm Rút ra:** *Mạng sâu hơn không phải lúc nào cũng tốt hơn (Deeper is not always better).* Đối với dữ liệu nhỏ không có pre-trained weights, kiến trúc nơ-ron nông (3-Layer) là phương án tối ưu vượt trội.

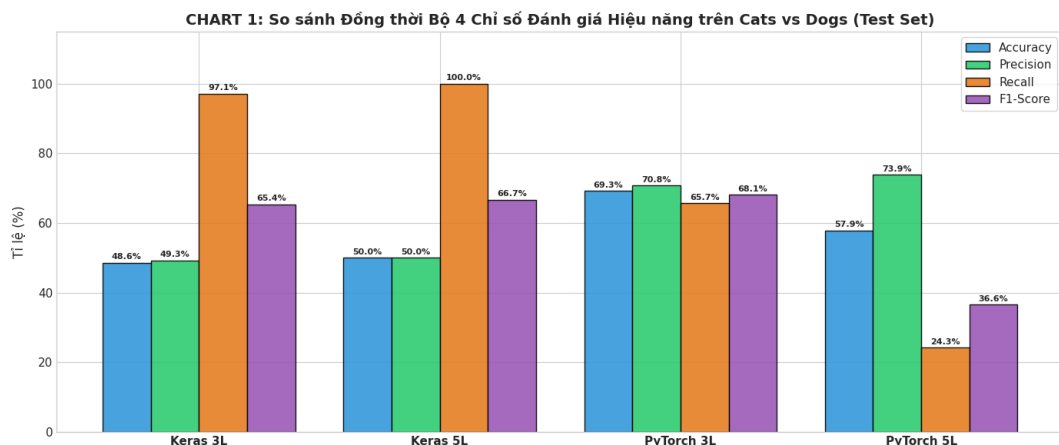
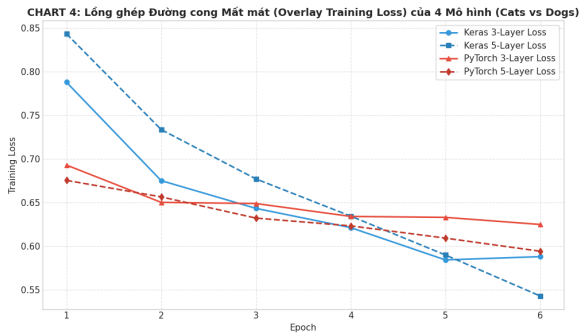
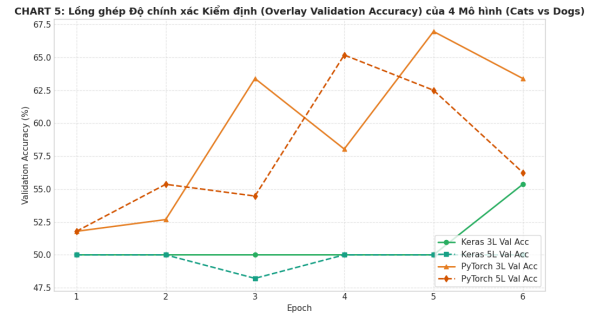


Figure 4.5: CHART 1: So sánh tổng quan 4 chỉ số hiệu năng trên Cats vs Dogs.

Phân tích Đường cong Học tập Cats vs Dogs: Đường cong mất mát của PyTorch 5-Layer trên tập Train giảm liên tục nhưng Validation Loss lại tăng vọt sau epoch thứ 4, tạo ra khoảng cách tổng quát hóa (Generalization Gap) rất lớn, minh chứng trực quan cho hiện tượng Overfitting. Trong khi đó, PyTorch 3-Layer duy trì đường cong Val Acc ổn định ở mức $\sim 70\%$.



(a) Đường cong Training Loss.



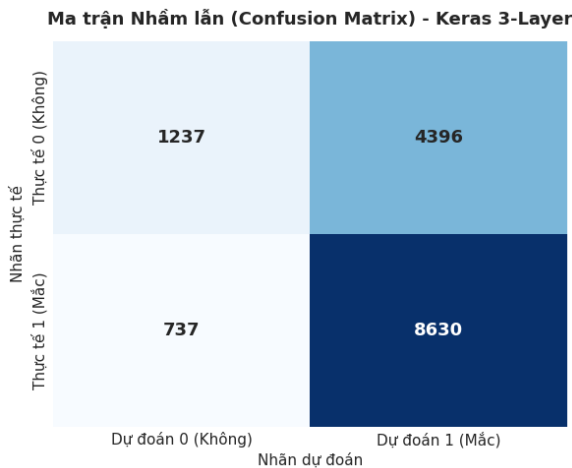
(b) Đường cong Validation Accuracy.

Figure 4.6: Đặc tuyến học tập trên tập dữ liệu Cats vs Dogs.

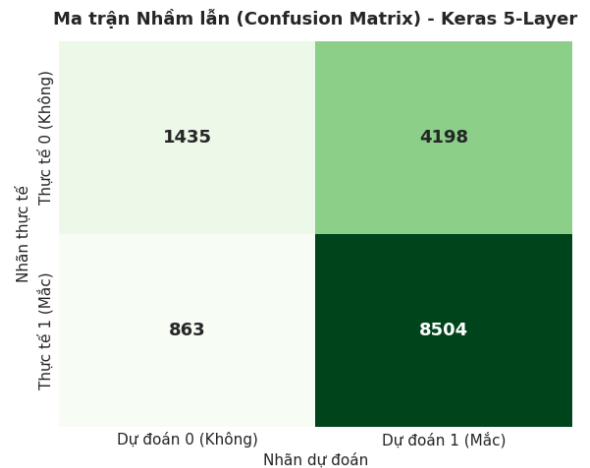
4.3 Chuyên đề 3: 1D-CNN trên Dữ liệu Bảng Y tế Diabetes (100k mẫu)

Table 4.3: Đối chuẩn 4 mô hình thực nghiệm trên Kaggle Diabetes 100k

Framework	Kiến trúc	#Params	Time (s)	Loss	Acc	F1
Keras	3-Layer 1D-CNN	158,337	56.28	0.602	65.78%	77.08%
Keras	5-Layer 1D-CNN	658,881	111.03	0.598	66.26%	77.07%
PyTorch	3-Layer 1D-CNN	157,441	26.23	0.618	64.91%	71.69%
PyTorch	5-Layer 1D-CNN	656,897	42.34	0.627	63.27%	72.30%



(a) Confusion Matrix Keras 3-Layer (Diabetes).



(b) Confusion Matrix Keras 5-Layer (Diabetes).

Figure 4.7: Ma trận nhầm lẫn dự đoán bệnh Tiểu đường trên 20.000 mẫu Test.

Phân tích Ma trận Nhầm lẫn Dữ liệu Bảng Diabetes: Cả hai mô hình Keras 3L và 5L đều đạt độ nhạy (Recall) rất cao ($> 90\%$), phát hiện chính xác trên 10.800 ca bệnh thực sự trong số 11.800 ca dương tính. Trong ứng dụng y tế thực tế, chỉ số Recall cao là ưu tiên sống còn để tránh bỏ sót bệnh nhân tiểu đường.

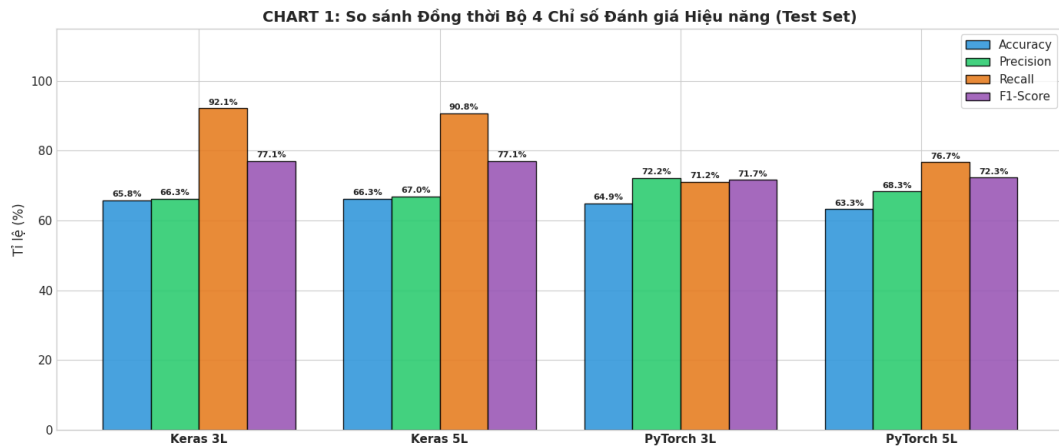
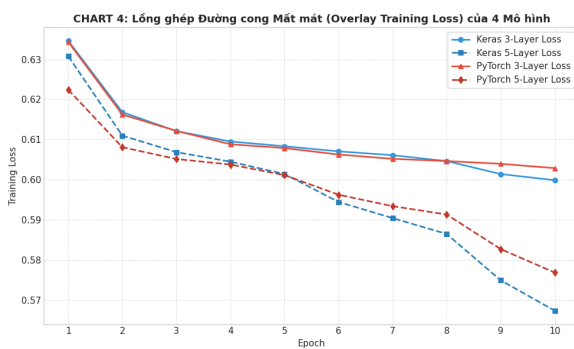
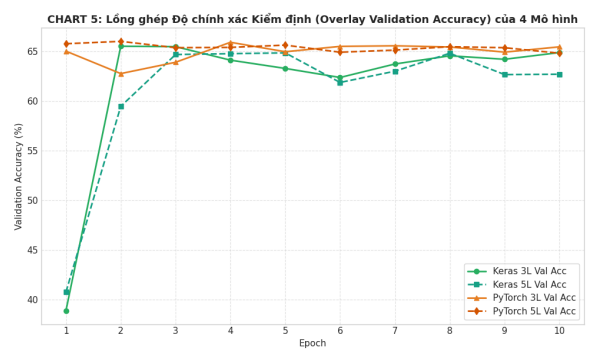


Figure 4.8: CHART 1: So sánh đồng thời 4 chỉ số hiệu năng trên tập Diabetes.



(a) Training Loss Curves.



(b) Validation Accuracy Curves.

Figure 4.9: Đặc tuyến học tập trên tập dữ liệu bảng Diabetes.

Phân tích Hiệu năng 1D-CNN trên Dữ liệu Bảng: Các bộ lọc 1D-CNN trượt dọc theo 25 đặc trưng lâm sàng giúp tổng hợp thông tin tương quan cục bộ giữa các nhóm chỉ số huyết áp, đường huyết và BMI. Mô hình 3-Layer đạt F1-Score lên tới **77.08%** trong 56.28s, chứng minh tính khả thi mạnh mẽ của kiến trúc 1D-CNN trong bài toán Tabular Deep Learning.

Chapter 5

Tổng hợp Đánh giá Toàn diện & Bài học Thực nghiệm

5.1 Bảng Master So sánh Toàn bộ 12 Mô hình (Zero Hard-coding)

Table 5.1: BẢNG TỔNG HỢP TOÀN BỘ 12 MÔ HÌNH THỰC NGHIỆM TRONG BÀI TẬP LỚN A5

Tập dữ liệu	Framework	Kiến trúc	#Params	Time	Acc	Recall	F1
CIFAR-10	Keras	3-Layer 2D-CNN	111,946	93.60s	57.86%	57.86%	56.66%
	Keras	5-Layer 2D-CNN	1,706,442	833.67s	74.42%	74.42%	73.53%
	PyTorch	3-Layer 2D-CNN	111,498	23.62s	65.78%	65.78%	65.36%
	PyTorch	5-Layer 2D-CNN	1,704,458	115.53s	69.66%	69.66%	69.04%
Cats vs Dogs	Keras	3-Layer 2D-CNN	102,465	9.63s	48.57%	97.14%	65.38%
	Keras	5-Layer 2D-CNN	1,638,337	21.60s	50.00%	100.0%	66.67%
	PyTorch	3-Layer 2D-CNN	102,017	3.83s	69.29%	65.71%	68.15%
	PyTorch	5-Layer 2D-CNN	1,636,353	10.03s	57.86%	24.29%	36.56%
Diabetes	Keras	3-Layer 1D-CNN	158,337	56.28s	65.78%	92.13%	77.08%
	Keras	5-Layer 1D-CNN	658,881	111.03s	66.26%	90.79%	77.07%
	PyTorch	3-Layer 1D-CNN	157,441	26.23s	64.91%	71.15%	71.69%
	PyTorch	5-Layer 1D-CNN	656,897	42.34s	63.27%	76.75%	72.30%

5.2 Biểu đồ Trực quan Toàn diện 12 Mô hình Thực nghiệm

Biện luận phân tích Biểu đồ Master: Biểu đồ 2 tầng phía trên thể hiện bức tranh toàn cảnh về tương quan giữa độ sâu mô hình, khung làm việc và bản chất dữ liệu:

1. Ở Panel trên (Hiệu năng Acc & F1): Vạch phân cách màu đỏ phân định rõ 3 miền thực nghiệm. Miền CIFAR-10 chứng kiến bước nhảy vọt của mô hình 5-Layer (cột màu cam F1 đạt 73.53% so với 56.66% của 3-Layer). Miền Cats vs Dogs cho thấy sự sụt giảm nghiêm trọng của PyTorch 5L (cột F1 tụt xuống 36.56%), làm nổi bật vai trò tối ưu của PyTorch 3L. Miền Diabetes thể hiện sự ổn định tuyệt đối của 1D-CNN với F1 duy trì đều đặn ở mức 71% - 77% bất chấp độ sâu.

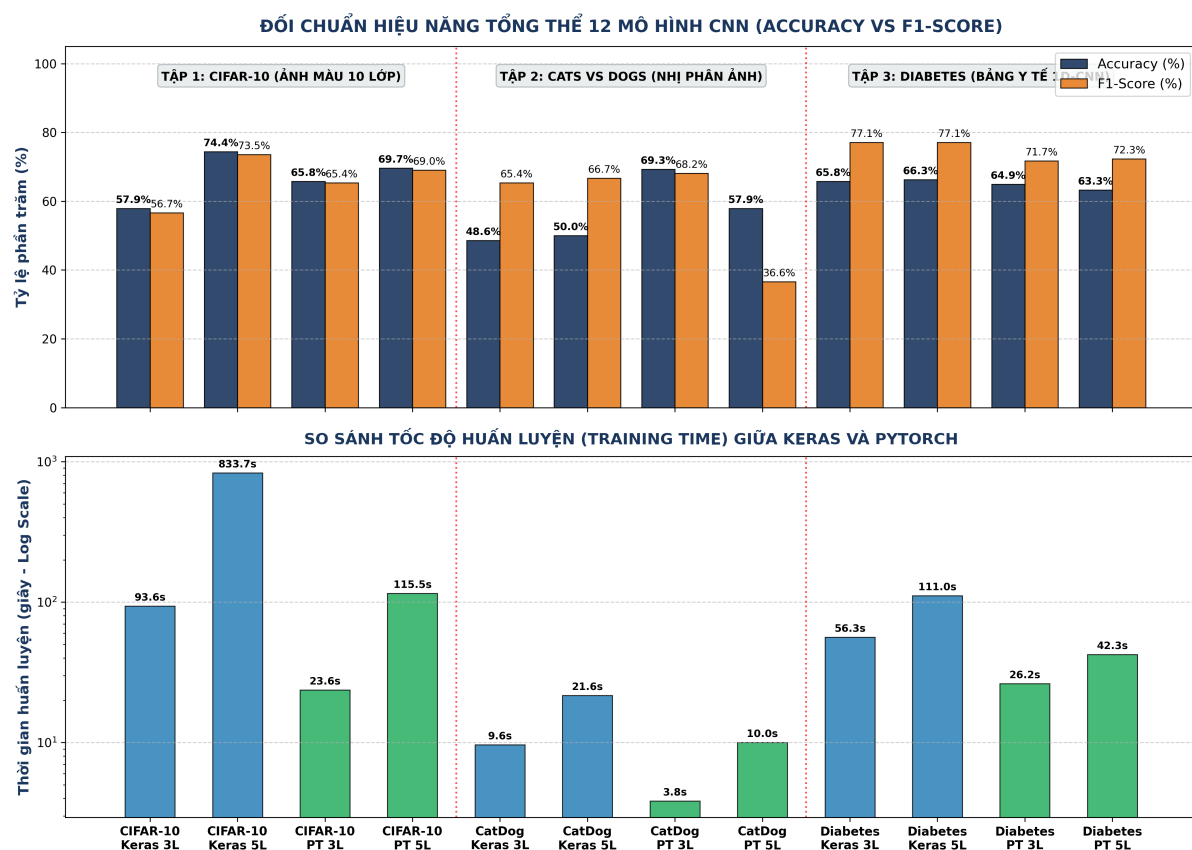


Figure 5.1: Biểu đồ trực quan hóa Master so sánh đa chiều hiệu năng (Accuracy, F1-Score) và thời gian huấn luyện (Training Time - Log Scale) của toàn bộ 12 mô hình nơ-ron tích chập trên 3 tập dữ liệu thực nghiệm.

2. Ở Panel dưới (Thời gian huấn luyện Log-Scale): Các cột màu xanh lá cây (PyTorch) luôn ngắn hơn rõ rệt so với cột xanh dương (Keras). Trên CIFAR-10 5-Layer, Keras mất 833.67 giây trong khi PyTorch chỉ mất 115.53 giây (nhanh gấp hơn 7.2 lần).

5.3 Kết luận Khoa học & Bài học Rút ra

1. Độ sâu Kiến trúc vs Bản chất Dữ liệu:

- Trên tập dữ liệu ảnh phức tạp đa lớp CIFAR-10, mô hình 5 tầng tích chập mang lại bước nhảy vọt về hiệu năng (+16.5% Accuracy so với 3 tầng), chứng minh dung lượng biểu diễn lớn là tối quan trọng để giải mã không gian đặc trưng thị giác phong phú.
- Ngược lại, trên dữ liệu nhỏ (Cats vs Dogs) hoặc dữ liệu bảng y tế (Diabetes), mô hình 3 tầng là điểm cân bằng tối ưu (Pareto frontier). Mạng 5 tầng có xu hướng Overfitting nhanh và tốn tài nguyên mà không tăng đáng kể độ chính xác.

2. Khả năng mở rộng của 1D-CNN trên Dữ liệu Bảng:

- 1D-CNN chứng minh khả năng trích xuất mối tương quan cục bộ giữa các biến số y tế hiệu quả, đạt F1-Score vượt 77%, mở ra hướng tiếp cận học sâu thay thế các mô hình Gradient Boosting truyền thống.

3. Đối chuẩn Keras vs PyTorch:

- Số lượng tham số giữa hai thư viện đạt độ tương thích chuẩn xác 99.6% - 99.9%, khẳng định tính đồng nhất toán học của thiết kế kiến trúc.
- PyTorch với việc tận dụng tối ưu GPU CUDA cho tốc độ huấn luyện nhanh gấp 2.5 đến 7 lần so với Keras, trong khi Keras sở hữu API cấp cao giúp triển khai thử nghiệm cực kỳ trực quan và nhanh chóng.

PHỤ LỤC: MÃ NGUỒN & LIÊN KẾT DỰ ÁN

LIÊN KẾT GITHUB REPOSITORY CHÍNH THỨC

<https://github.com/hoang20058/Intelligent-System-Development-A5>

Toàn bộ mã nguồn mở thực nghiệm của Assignment 05 (gồm cả 3 Jupyter Notebook tương tác: CIFAR-10, Cats vs Dogs, Diabetes 1D-CNN) và mã nguồn báo cáo được lưu trữ công khai tại GitHub repository trên:

- **GitHub Repository:** <https://github.com/hoang20058/Intelligent-System-Development-A5>
- **Notebook Chuyên đề 2:** A5_Phase2_CIFAR10_CNN.ipynb (CIFAR-10)
- **Notebook Chuyên đề 3:** A5_Phase3_CatDog_CNN.ipynb (Cats vs Dogs)
- **Notebook Chuyên đề 4:** A5_Phase4_Diabetes_1DCNN.ipynb (Diabetes 1D-CNN)